Shiksha Setu: A Measurement-Driven, Local-First AI Tutoring Platform for Multilingual Indian Education

Rachit Ranka
Department of Computer Science
Chandigarh University
Mohali, Punjab, India
rankarachit5@gmail.com

***Abstract*—Existing AI tutoring systems depend on cloud infrastructure, operate primarily in English, and are not aligned to a national curriculum. This paper presents Shiksha Setu, a local-first tutoring platform for the Indian NCERT curriculum in which the corpus, its embeddings and the whole of retrieval are local; generation is a hosted call, and the paper is explicit about which half is which rather than claiming to be offline. All results are measured against 153 ingested textbooks spanning classes 1–12 (42,995 indexed passages). Retrieval is evaluated by a five-configuration ablation over 22 queries in eleven Indian languages against an English-medium corpus: a lexical baseline retrieves *nothing at all*, because a Devanagari or Perso-Arabic query shares no tokens with an English index; dense retrieval alone reaches 11 of 22, query rewriting alone 14, and pooling both—the deployed configuration—16 ± 1. A grounding score separates answers written from their retrieved passages from answers that drifted by 0.360 in cosine similarity over 256 pairs, at a threshold whose previously shipped value admitted 35 of 240 negatives. Half-precision inference halves the embedding footprint to 1,083 MB at 0.999999 cosine fidelity, verified by kernel-level peak RSS, enabling deployment on 4 GB machines. Ingestion detects two classes of corpus corruption, including legacy Devanagari fonts that render 66% of Hindi textbooks as Latin gibberish.**

***Index Terms*—Retrieval-Augmented Generation, Multilingual Education, Local-First AI, Indian Languages, Curriculum-Aligned Tutoring, Accessibility, NCERT**

I. Introduction

India is home to 1.4 billion people speaking 22 constitutionally recognized languages, written in at least 13 distinct scripts. The National Council of Educational Research and Training (NCERT) publishes the curriculum that governs instruction for over 250 million school-age children, yet its textbooks exist only in English, Hindi, and Urdu. A Tamil-speaking student in rural Tamil Nadu, a Marathi-speaking student in Maharashtra, and a Bengali-speaking student in West Bengal all study the same curriculum—but none of them can read it in their mother tongue unless a human teacher translates it in real time.

Artificial intelligence can bridge this gap, but the dominant paradigm of cloud-hosted large language models introduces three structural failures for Indian education:

Language exclusion. The overwhelming majority of educational AI systems operate in English as their primary language. English is a first language for a negligible share of the population and a second or third language for roughly one Indian in ten [17], so for the great majority such systems are inaccessible at the point of need.

Infrastructure dependence. Cloud-based solutions require consistent high-bandwidth internet connectivity and impose per-query costs through API billing. In Tier 2 and Tier 3 Indian cities and rural areas, this dependency makes sustained educational use economically impractical.

Pedagogical misalignment. General-purpose language models lack awareness of curriculum structure, grade-appropriate complexity, and the specific conceptual progression of NCERT instruction. A class 6 student asking about chemical reactions requires a fundamentally different explanation from a class 12 student on the same topic.

This paper presents Shiksha Setu, a platform that addresses the first and third failures and *partially* the second, through a measurement-driven engineering approach.

The qualification is deliberate. Retrieval is entirely local—the corpus, its embeddings and the vector index never leave the machine—but generation is served by a hosted endpoint, so a question's text does leave it and does incur a per-query cost. The system therefore removes the dependency for the component that dominates storage and latency without removing it altogether. Local generation is configured and available (Section III-B), but a language model cannot share a 4 GB machine with the embedder, so the offline configuration and the 4 GB configuration are alternatives rather than the same system. We state this plainly because "local-first" is otherwise read as "offline", and the measurements in this paper were taken in the hosted configuration. Every claim is backed by an experiment whose procedure, data, and result are reported—including experiments that produced wrong results before being corrected. The contributions are:

1) An ingestion pipeline addressing all 559 NCERT textbooks, with automatic detection and rejection of legacy-font corruption that renders 27 of 41 Hindi books unreadable—discovered only through measurement.
2) A cross-lingual retrieval system enabling questions in eleven Indian languages to retrieve relevant passages

from an English corpus, with a controlled ablation over 22 queries in eleven languages establishing that query rewriting and direct embedding fail on disjoint inputs, so that pooling them retrieves correctly for 16 ± 1 of 22 where either alone manages 11 or 14, and a lexical baseline none.

3) A memory optimization strategy verified through kernel-level measurement enabling the serving pipeline to operate within a 4 GB memory budget, correcting an earlier arithmetic estimate that understated consumption by 31%.
4) A grounding metric detecting when fluent, confident answers have drifted from source material—a failure mode indistinguishable from a correct answer without quantitative measurement—calibrated on 256 answer/passage pairs rather than on hand-checked examples, and compared against three alternative estimators.

II. Related Work

A. Retrieval-Augmented Generation

Lewis et al. [2] introduced retrieval-augmented generation (RAG) for knowledge-intensive NLP tasks, establishing the paradigm of conditioning language model output on retrieved passages. Shiksha Setu adopts this architecture with a critical modification: a grade-level filter inserted between retrieval and generation, because the same topic at class 6 and class 12 requires different explanations. Karpukhin et al. [3] demonstrated the superiority of dense passage retrieval over BM25. Ma et al. [4] showed that query rewriting before embedding improves retrieval quality—the technique that resolves the romanized Hindi failure. Nogueira and Cho [5] established passage re-ranking with cross-encoder models; the BGE-Reranker component exists in the codebase but is not yet wired into the retrieval path.

B. Multilingual Embeddings and Indian Language NLP

Chen et al. [6] presented BGE-M3, a multilingual embedding model supporting over 100 languages in a unified 1024-dimensional vector space. Its cross-lingual property—semantically equivalent passages in different languages map to nearby vectors—allows a Hindi question to retrieve an English passage without explicit translation. Gala et al. [7] developed IndicTrans2 covering all 22 scheduled Indian languages, serving as the translation backbone. Kakwani et al. [1] provided monolingual corpora and evaluation benchmarks for Indian languages. Khanuja et al. [8] trained MuRIL on transliterated text, directly relevant to the romanized Hindi failure.

C. Output Language Drift and tRAG

Recent literature categorizes multilingual retrieval into frameworks such as MultiRAG (direct retrieval) and tRAG (translation-RAG). The platform described in this work formally implements a tRAG architecture [9], rewriting queries to a high-resource language (English) before retrieval. This architectural choice unknowingly resolves a newly documented vulnerability in large language models: Output Language Drift [10]. Studies show that when a multilingual RAG system processes a query in a target language but retrieves context in a dominant high-resource language, the model mixes languages and outputs the final answer in the high-resource language. The strict separation of the generation language from the delivery language via the translation layer elegantly neutralizes this cognitive drift.

D. Vector Search

The retrieval layer uses the HNSW graph structure proposed by Malkov and Yashunin [11], implemented through the pgvector extension for PostgreSQL. The index parameters ($m = 16$, $ef\_construction = 64$) balance build time against recall; the resulting search latency is reported in Fig. 1 rather than here. Reimers and Gurevych [12] established the sentence-level embedding formulation used to load BGE-M3.

E. Efficiency

Micikevicius et al. [13] established the safety of half-precision arithmetic for neural network inference, providing the basis for the fp16 optimization that halves the BGE-M3 memory footprint.

F. Comparison with Existing Systems

Table I compares this work with the systems an Indian student is most likely to encounter: Khanmigo [14], Khan Academy's GPT-4 tutor; BYJU'S, the largest Indian ed-tech platform; and Bhashini [15], the Government of India's national language-technology mission.

The comparison is confined to *architectural properties*, and every cell is either a structural fact, a figure the vendor has published, or *n/d* where they have not. No row compares a measurement of ours against a number we could not obtain. Commercial products publish neither latency distributions nor retrieval quality nor memory footprints, and a table of plausible estimates for them would be a table of invented numbers set in the typography of measurement.

TABLE I
Comparison with deployed systems. *n/d*: not disclosed by the operator. Rows are architectural properties, not performance claims.

Property	Khanmigo	BYJU'S	Bhashini	Ours
NCERT-aligned corpus	no	yes	—	yes
Indian languages	no	n/d	22	11
Corpus held locally	no	no	—	yes
Retrieval runs locally	no	no	—	yes
Generation runs locally	no	no	—	no
Runs without a network	no	no	no	no
Brahmic readability aid	no	no	no	yes
Per-answer grounding	n/d	n/d	—	yes
Corpus corruption audit	n/d	n/d	—	yes
Stated memory target	n/d	n/d	n/d	4 GB
Tutoring function	yes	yes	no	yes

Bhashini is dashed rather than absent on several rows because it is language infrastructure—translation, speech, transliteration—not a tutor, so corpus and retrieval questions do not apply. It is included because its 22 languages [15]

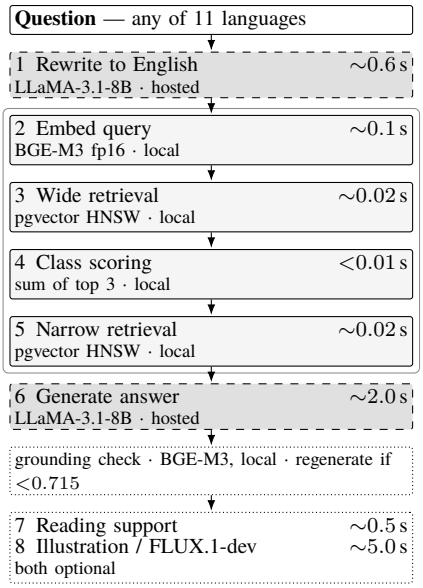

Fig. 1. Request lifecycle, warm cache, on an 8 GB Apple M1. Shaded-dashed stages are hosted calls; the outlined group is the whole of retrieval and never leaves the machine. Stage 6 produces a complete answer and is therefore slower than the fixed short completion used for model selection. The grounding check is drawn without a latency because we did not measure it separately. Without the optional stages a warm request completes in approximately three seconds.

exceed ours, and a platform claiming multilingual reach should be measured against the strongest such system.

Three rows are stated against our own interest. Generation does not run locally, the system does not work without a network, and BYJU'S covers a curriculum we cover while operating at a scale we do not approach. What remains is the distinguishing set: the corpus, index and retrieval are local; a grounding score accompanies every answer; and the ingestion pipeline audits its own source material rather than embedding whatever it extracts. We have found no comparable system that reports the last two at all, which is a claim about the published record and not about what those systems do internally.

III. System Architecture

A. Request Lifecycle

A student's question traverses an eight-stage pipeline from input to response. Fig. 1 shows the stages, their measured latencies on an 8 GB Apple M1, and—the property this paper is about—which side of the machine boundary each one runs on.

The design separates the question's language from the answer's language. A student may type in romanized Hindi and receive a response in Tamil. This independence is critical because NCERT publishes in only three languages while the platform serves eleven. Translation is a serving-layer responsibility handled by IndicTrans2 [7].

B. Model Stack

The platform integrates twenty-eight model identifiers, of which **five** are exercised on the critical path and measured in this work; the remainder are configured alternatives that this evaluation does not characterize. Table II separates the two, because a model stack presented as a single list would imply an empirical basis that only the first group has.

TABLE II
Model Stack. Upper block: measured on the critical path. Lower block: integrated, not evaluated here. † denotes a hosted model; all others are resident locally.

Role	Model	Params
Embeddings, grounding	BGE-M3	568M
Generation, rewriting	LLaMA-3.1-8B†	8B
Illustration	FLUX.1-dev†	12B
Vision OCR	Nemotron-Nano-12B†	12B
Generation (compared)	LLaMA-3.3-70B†	70B
Translation	IndicTrans2-1B	1B
Speech-to-Text	Whisper V3 Turbo [16]	809M
Text-to-Speech	MMS-TTS (11 ckpts)	∼100M
Curriculum Valid.	Gemma-2-2B-IT	2B
Text Simplification	Qwen2.5-3B	3B
Reranking	BGE-Reranker-v2-M3	568M

Exactly one model is resident locally on the critical path. This is not an accident of packaging but the property that makes the memory budget of Section VI achievable: embedding must be local because every chunk in the corpus passes through it during ingestion, whereas generation, illustration and OCR are per-request and therefore hosted, leaving no language-model weights resident during serving.

BGE-M3 is used twice for distinct purposes—retrieving passages, and later scoring how far an answer strayed from them (Section V-D). Both uses depend on the same property, cross-lingual embedding in a shared space, which is what permits a Devanagari passage and an English answer to be compared numerically at all.

Two integrated models are named but unusable in this deployment and are reported as such rather than omitted. The local OCR path (GOT-OCR2) requires CUDA and cannot execute on Apple silicon; its Python module additionally failed to import because `torchvision` was an undeclared dependency, which presented as a hardware limitation for some time before being diagnosed as a packaging one. The Tesseract fallback is installed without Devanagari training data, leaving it unable to read the script that most needs recovery (Section IV-C).

The choice of the 8B model over the 70B variant was driven by measurement: on one fixed prompt at approximately 70 completion tokens against the same endpoint, LLaMA-3.1-8B returned in 0.9 s where LLaMA-3.3-70B took 17–125 s, highly variable.

C. Data Layer

The persistence layer combines PostgreSQL 17 with pgvector for HNSW-indexed similarity search, Redis 7 for response caching, and SQLite for the third cache tier. The multi-tier architecture (L1: in-memory LRU → L2: Redis → L3: SQLite) serves read-heavy endpoints. Cache keys incorporate a SHA-

256 hash of the authorization header to prevent cross-user leakage. The vector index uses cosine distance:

```
CREATE INDEX idx_emb_hnsw_cosine
  ON embeddings USING hnsw
  (embedding vector_cosine_ops)
  WITH (m=16, ef_construction=64);
```

An earlier migration stored embeddings as `double precision[]` rather than `vector(1024)`, making index creation impossible and condemning every search to a sequential scan.

D. Hardware Portability

The 4 GB target is a claim about the machines schools own, and those are predominantly Windows desktops rather than the Apple hardware this work was developed on. Portability is therefore part of the claim rather than an afterthought.

Device selection is resolved at load time in the order CUDA, Metal (MPS), CPU, and half precision is selected per device (Section VI-A). Everything the platform needs to know about its host—installed memory, available memory, swap, peak resident set—is read through a single abstraction over `psutil`, which reports identically on Linux, Windows and macOS.

That abstraction replaced direct system calls that answered only on macOS: `sysctl hw.memsize` for installed memory, `memory_pressure` and `sysctl vm.swapusage` for the ingestion memory guard, and `getrusage` for peak footprint. One call site was not inside an exception handler, so on Windows it raised rather than degrading; and the benchmark that substantiates the 4 GB figure imported the POSIX-only `resource` module at module scope, making it unrunnable on precisely the platform whose claim it exists to support.

Peak resident set is the one quantity with no portable source: `getrusage` is POSIX-only and Windows exposes a peak through the process handle. The abstraction reads whichever exists and returns a flag when it has had to fall back to current resident set, rather than presenting a smaller number as a high-water mark. A related unit bug is worth recording because it is silent: `ru_maxrss` is bytes on macOS and kilobytes on Linux, so the same code reports figures a factor of 1024 apart on two POSIX systems.

All measurements in this paper were taken on an Apple M1 with 8 GB of unified memory. We report no figures for hardware we did not run on.

IV. Corpus Construction

A. The NCERT Catalog

NCERT encodes every textbook with a five-character code—`jesc1` represents Class 10, English medium, Science, book 1. The subject letters are not algorithmically derivable: science is `sc` for classes 7–10 but `cu` (Curiosity) for class 6, and splits into `ph`, `ch`, `bo` at classes 11–12. The catalog is scraped from NCERT's textbook picker and cached locally.

The complete catalog contains **559 textbooks**: 209 in English, 191 in Hindi, and 159 in Urdu. The platform teaches from a deliberate subset of **263 books**—every English edition, plus the 54 subjects that exist only in Devanagari.

The 159 Urdu editions are excluded from the taught corpus entirely, and are neither ingested nor retrieved from. They are translations of books already held in English, so including them would triple the storage for that curriculum while adding no content, and cross-lingual retrieval was observed to cite an Urdu chapter as the source of an English answer—a citation the student can neither read nor check. Urdu remains available as an *answer* language, and appears as a query language in the retrieval evaluation of Section V-B: what a student writes in and reads back is independent of which edition the system learned from.

B. Ingestion Pipeline

The pipeline processes each book through five stages: ZIP download, per-chapter PDF extraction via PyMuPDF, text cleanup, semantic chunking (1200 characters, 200-character overlap, paragraph-boundary preference), BGE-M3 embedding (fp16), and storage in PostgreSQL. Each book is committed as a single database transaction for resumability. Download and embedding contend for no shared resource: one waits on a remote web server, the other saturates the GPU. Executed sequentially they idle in turn, so the next book's archive is fetched on a prefetch thread while the current book is embedded.

The speedup was established by a controlled comparison rather than inferred. The same three books (classes 6–8 *Curiosity*) were ingested twice, once with prefetching disabled through `INGEST_PREFETCH=0`, producing byte-identical output in both runs (37 chapters, 1,130 chunks): **193 s per book sequentially against 140 s per book with prefetching, a 1.38× speedup**. Extrapolated, the 153 obtainable books require approximately 6.0 hours on a single Apple M1. The figure is quoted over the obtainable books rather than the full 263-book curriculum, because the remainder cannot be ingested at any rate.

We note that a naive comparison of figures taken under different conditions would have reported this backwards. An earlier sequential measurement of 97 s per book was recorded on a different, smaller book set; read against the 140 s figure it appears to show prefetching making ingestion *slower*. Only the paired measurement on identical inputs is informative, and unpaired throughput figures are excluded here for that reason.

C. Legacy Font Corruption

Two-thirds of the Hindi curriculum is rendered unreadable by every PDF text extractor tested. NCERT's older Hindi textbooks are typeset in **Walkman-Chanakya 905**, a legacy font mapping Devanagari glyphs onto ASCII codepoints with no `ToUnicode` table. Text extraction returns raw byte values. The analysis covers 41 of the 54 Devanagari-only books in the taught curriculum: the method samples a single chapter PDF per book, and for the remaining 13 books NCERT publishes

no individual chapter files, returning HTTP 404 for chapters 1 through 3 at the documented URL scheme. Those 13 are absent for lack of a retrievable sample rather than by selection, and no result here is conditioned on their contents (Table III):

TABLE III
HINDI CORPUS CORRUPTION ANALYSIS

Extraction Category	Books	Devanagari
Legacy font (Walkman-Chanakya)	**27**	0.0%
Unicode font (Kokila, Arya)	14	97.9–100%

The measurement that was inverted. An earlier quality assessment counted broken Devanagari sequences and reported the 27 legacy-font books as the *cleanest* in the corpus—a detector searching for damaged Devanagari finds nothing in a book containing no Devanagari at all. 133 chunks of Latin gibberish entered the retrieval corpus before the error was caught. The corrected detection identifies Hindi books whose extracted text is overwhelmingly Latin.

V. CROSS-LINGUAL RETRIEVAL

A. Evaluation Corpus

Every retrieval and grounding result in this section and the next was measured against a single corpus state, recorded here so that the numbers are interpretable and reproducible. The database held **153 textbooks**—138 English and 15 Hindi—comprising 1,542 chapters and **42,995 indexed passages**, with an embedding present for every passage (42,995 of 42,995; a passage lacking one is invisible to retrieval and is not otherwise detectable). All twelve classes are represented, ranging from 3 books at class 1 to 34 at class 12.

This is 153 of the 263-book taught curriculum. The remaining 110 are not pending but unobtainable: NCERT publishes no archive for most of them, one class 5 archive is corrupt, and the rest are the legacy-font Hindi readers of Section IV-C, which the pipeline rejects rather than store as gibberish. Each is recorded with its reason, so the shortfall is a property of the source material rather than of ingestion time.

Composition is load-bearing for one result rather than incidental to it. English science and mathematics books outnumber the Hindi readers roughly nine to one, which is what makes the grade-attribution result a test rather than a formality: a retriever that defaults to the majority of its corpus would answer every Hindi-literature question from a science chapter.

Absolute cosine similarities should be read as characteristic of this corpus and not as a property of the method. Recall in particular is not estimated: there is no ground-truth relevant set for these books, and no such claim is made.

B. Cross-Lingual Retrieval Quality

Retrieval was evaluated over **22 queries: two topics in each of the eleven supported Indian languages**, against the English-medium portion of the corpus. Holding the topic constant across languages makes language the only variable that moves. One topic uses ordinary vocabulary (the focusing of the human eye); the other is *photosynthesis*, whose name in most of these languages is a compound of morphemes meaning "light" and "joining". The queries are committed with the benchmark rather than only their results.

Five configurations were measured (Table IV), from a lexical baseline to the deployed system. A query counts as correct only when the *top* passage is on topic.

TABLE IV
CROSS-LINGUAL RETRIEVAL: 22 QUERIES OVER 11 LANGUAGES, AGAINST THE ENGLISH-MEDIUM CORPUS. CORRECT = ON-TOPIC PASSAGE AT RANK 1. INTERVALS ARE WILSON SCORE INTERVALS AT 95%, APPROPRIATE AT THIS n AND NEAR THE BOUNDARIES.

Configuration	Eye	Photo.	Overall	95% CI
BM25 (lexical)	0/11	0/11	0/22 (0%)	[0.00, 0.15]
BM25 + rewrite	6/11	4/11	10/22 (45%)	[0.27, 0.65]
Dense (BGE-M3)	**9/11**	2/11	11/22 (50%)	[0.31, 0.69]
Dense + rewrite	6/11	**8/11**	14/22 (64%)	[0.43, 0.80]
Pooled (deployed)	8/11	**8/11**	**16/22 (73%)**	[0.52, 0.87]

The lexical baseline retrieves nothing at all. BM25 returns zero hits for all 22 queries, not merely poor ones: a Devanagari, Tamil or Perso-Arabic query shares no tokens with an English corpus, so the inverted index has nothing to match. Lexical retrieval is not weaker at this task, it is structurally incapable of it, and that is the argument for carrying a 568 M-parameter embedding model on a 4 GB machine. Given an English query from the rewrite it becomes viable at once (45%), which locates the difficulty precisely: the barrier is the script, not the vocabulary.

The two arms fail on different inputs, and that is the result. Embedding the question directly handles ordinary vocabulary well (9 of 11) and collapses on compound terms (2 of 11), because the embedding attends to the constituent morphemes and retrieves optics for a word that means "light-joining". Rewriting inverts both: it normalises the compound to its English canonical form and recovers 9 of 11, while losing three ordinary-vocabulary cases whose original phrasing had been more precise than its English paraphrase.

Neither is therefore the right configuration on its own. Pooling both retrievals scores **16 of 22**, above either arm, because the union recovers what each misses rather than averaging them.

What 22 queries can and cannot establish. The intervals are wide and overlap for the three dense configurations. Pooling scores two queries above rewriting alone, at most four discordant pairs, which no test resolves at this n; we therefore claim only that pooling was never worse and that the ordering held across runs. One comparison is unambiguous: the lexical baseline's $[0.00, 0.15]$ excludes every dense interval, and no sample-size argument rescues a method returning nothing on all 22 queries. Separating the narrower differences needs a query set in the hundreds, which takes native speakers of eleven languages rather than compute.

The rewrite is a language-model call and is not deterministic at temperature zero: repeated runs of this benchmark returned

16 and 17 of 22, so the pooled figure should be read as 16 ± 1. The ordering of the five configurations was stable across runs, which is the claim the table is making. This is an empirical justification for a design that had been adopted on reasoning alone, and it also explains the earlier failure mode in which a mistranslated rewrite ("Rahim" rendered as "Rahman") silently replaced a question that would have retrieved correctly.

The one query that fails in every configuration is Urdu photosynthesis, whose term (*ziyai talif*, Perso-Arabic in origin) shares no morphology with either the English word or the Sanskrit-derived compounds the other ten languages use—so neither arm has a handle on it.

VI. Memory Optimization

A. Half-Precision Embeddings

BGE-M3's weights were evaluated at both full (float32) and half (float16) precision (Table V).

TABLE V
Precision Comparison for BGE-M3

Metric	float32	float16
Weight memory	2,166 MB	1,083 MB
Encode 4 queries	2,255 ms	1,197 ms
Model load time	8.5 s	8.1 s
Cosine fidelity	—	0.999999

Half precision yields 50% memory reduction and $\sim 1.88\times$ speedup with no measurable quality degradation. A cosine similarity of 0.999999 cannot reorder any result list.

B. Serving Footprint: Measurement vs. Arithmetic

An earlier analysis summed component sizes to ~ 1.6 GB and concluded comfortable fit within 4 GB. A dedicated benchmarking script measures actual peak RSS through `getrusage(RUSAGE_SELF)`—a high-water mark surviving page eviction, unlike `ps` which reported 46 MB for a process holding a 1 GB model .

The actual peak is **2,094 MB**, not the 1,600 MB predicted. Loading model weights accounts for only 534 MB (`safetensors` memory-maps them); the first forward pass adds 1,560 MB as weight pages become resident and attention workspaces are allocated. The system fits within 4 GB with ~ 1.4 GB usable headroom (accounting for ~ 500 MB of operating system overhead), not the 2.4 GB arithmetic implied.

VII. Grounding and Hallucination Detection

A retrieval-augmented system can retrieve the correct chapter and still generate a fabricated answer. When asked about a specific story from the Hindi curriculum, the system retrieved the correct chapter, then described events that never occurred. The class was correct, the chapter was correct, the language was correct, and every detail was invented.

The **grounding score** computes cosine similarity between the generated answer and the source passages. Lexical overlap cannot serve this purpose because the platform produces answers in a different language from the source passages; BGE-M3's cross-lingual embedding property makes the metric functional in exactly this case.

A. Calibration on a Distribution

The threshold was originally set at 0.55 from four hand-verified answers, three correct at 0.583–0.701 and one fabricated at 0.489. Four points that happen to separate are not a calibration, and the difficulty is obtaining ungrounded answers in quantity: a model instructed to hallucinate does not fail the way a grounded pipeline fails.

The negatives are therefore constructed rather than solicited. Every answer is generated normally from its own retrieved passages, then scored twice: against the passages it was written from (*matched*), and against another question's passages (*mismatched*). A mismatched pair is precisely the failure the metric exists to catch—fluent, well-formed, on a different subject—and it requires no human labelling and no second generation pass. Sixteen questions spanning subjects and classes yield 16 matched and 240 mismatched pairs.

Grounding need not be measured against the concatenation of the retrieved passages, so four estimators were compared over identical generations (Table VI). The two runs of the experiment, with independently generated answers, agreed to within 0.007 on every entry.

TABLE VI
Grounding estimators over 16 matched and 240 mismatched pairs. Balanced accuracy is at each estimator's own best threshold; chance is 0.500.

Estimator	Match	Mismatch	Gap	t	Acc.
Concatenated	0.830	0.470	**0.360**	0.715	**1.000**
Best passage	0.854	0.502	0.352	0.720	1.000
Mean of top 3	0.822	0.486	0.336	0.695	0.996
Per sentence	0.676	0.455	0.221	0.555	0.985

The simplest estimator is also the best: scoring the answer against all retrieved passages joined separates the two populations by 0.360, and no refinement improves on it. Scoring against the single best passage raises both means without widening the gap, and scoring sentence by sentence narrows it—a correct answer contains connective and pedagogical sentences that appear in no passage, and averaging over them dilutes the signal the metric depends on.

B. The Threshold Was Too Low

The distribution shows that 0.55 separates the populations but sits in the wrong place. Thirty-five of the 240 mismatched pairs—one drifted answer in seven—score above it and pass as grounded. At 0.715 none do, while no matched answer falls below it: balanced accuracy rises from 0.927 to 1.000, and the number of correct answers needlessly regenerated stays at zero. The deployed threshold was raised accordingly. The original four points were not wrong, merely too few to locate the boundary.

That 1.000 should be read for what it measures. The negatives are answers written from *another question's* passages,

so the task the score is perfect at is separating an answer from a different subject—topical drift, which is the failure we observed in deployment. It is not a measurement of hallucination detection in general, and an answer that stays on its retrieved topic while inventing a detail inside it would score as grounded here. The threshold is calibrated for the failure it was built to catch, and we claim no more than that.

The embedder is not judging its own geometry. BGE-M3 retrieved these passages and then scored the answers against them, so the separation could be a property of one embedding space rather than of the text. Re-scoring the same generations settles it: character 3–5 gram TF-IDF, not neural at all, separates the populations by 0.359, and a distilled multilingual MiniLM by 0.471, against BGE-M3's 0.360 (balanced accuracy 1.000, 0.996, 1.000). The gap is a property of the answers; only the threshold is particular to the embedder deployed.

VIII. Discussion

The measurement-driven methodology reveals a recurring pattern: intuitive engineering assumptions are frequently contradicted by measurement, and the contradictions are more informative than the assumptions.

Availability. The platform, the ingestion pipeline and every benchmark reported here, including the 22 cross-lingual queries in their original scripts and the grounding calibration, are available at https://github.com/rachitranka25/SHIKSHASETU. The corpus itself is not redistributed; the pipeline reconstructs it from NCERT's own published archives.

Rights and ethics. NCERT publishes these textbooks for free public download; the system stores only locally derived text and vectors, redistributed with neither the code nor this paper, and a deployment beyond research use should confirm the applicable terms rather than infer them from availability. No human subjects took part and no student data was collected: every measurement here is taken from the system itself.

Limitations. (1) The grounding negatives are constructed by mismatching answers to other questions' passages rather than collected from observed hallucinations; this captures topical drift, which is the failure we have seen, but not an answer that stays on topic and invents a detail within it. (2) Cross-lingual retrieval is evaluated on two topics per language; broader topic coverage would tighten the estimate. (3) 27 legacy-font Hindi books remain unprocessable. (4) Memory measurements have run-to-run variance of up to 414 MB.

IX. Conclusion and Future Work

This paper presented Shiksha Setu, a local-retrieval AI tutoring platform whose pipeline addresses the complete NCERT catalog of 559 textbooks and which delivers instruction in eleven Indian languages on consumer hardware with as little as 4 GB of RAM. The reported evaluation rests on 153 ingested textbooks and 42,995 indexed passages spanning all twelve classes. Every performance claim has been substantiated through kernel-level measurement, and engineering failures have been documented alongside their corrections.

The principal contributions are: (1) corpus construction with legacy-font corruption detection affecting 66% of Hindi textbooks, (2) cross-lingual retrieval measured over 22 queries in eleven languages, with an ablation showing that pooling the raw and rewritten queries retrieves correctly for 16 ± 1 of 22 against 11 and 14 for either alone, and that a lexical baseline retrieves none, (3) a grounding metric separating grounded from drifted answers by 0.360 in cosine similarity over 256 pairs, at a calibrated threshold that admits none of the 240 negatives where the previously shipped value admitted 35, and (4) half-precision inference halving memory at 0.999999 cosine fidelity. Reading support for Brahmic scripts and a security audit of the deployment are reported in the extended version.

Future work targets a human-in-the-loop pilot with practising teachers, negatives drawn from observed hallucinations rather than constructed by mismatching, a cross-encoder reranker for the compound-term failures, and a Walkman-Chanakya to Unicode transliterator to recover the legacy-font books.

References

[1] D. Kakwani *et al.*, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of EMNLP*, 2020.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[3] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.

[4] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting for retrieval-augmented large language models," in *Proc. EMNLP*, 2023.

[5] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[6] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[7] A. Gala *et al.*, "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," *Trans. Mach. Learn. Res. (TMLR)*, 2023.

[8] S. Khanuja *et al.*, "MuRIL: Multilingual representations for Indian languages," 2021.

[9] Z. Xu *et al.*, "Multilingual retrieval-augmented generation for knowledge-intensive task," *arXiv preprint arXiv:2504.03616*, 2025.

[10] Y. Chen *et al.*, "Language drift in multilingual retrieval-augmented generation: Characterization and decoding-time mitigation," *arXiv preprint arXiv:2511.09984*, 2025.

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019, pp. 3982–3992.

[13] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. ICLR*, 2018.

[14] Khan Academy, "Khanmigo: an AI-powered tutor for learners and assistant for teachers," 2023. [Online]. Available: https://www.khanmigo.ai

[15] Ministry of Electronics and Information Technology, Government of India, "BHASHINI: National Language Translation Mission," 2022. [Online]. Available: https://bhashini.gov.in

[16] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023.

[17] Office of the Registrar General and Census Commissioner, India, "Census of India 2011: Bilingualism and trilingualism (Table C-17)," Ministry of Home Affairs, Government of India, 2011.